

SOL

Programming Language

Language and Toolchain Design Specification

A language optimized for human-AI co-development

Concept Design v0.1

August 5, 2026

STATUS: DESIGN EXPLORATION - NOT AN IMPLEMENTED OR STANDARDIZED LANGUAGE

Document Control

Field	Value
Document	Sol Programming Language - Language and Toolchain Design Specification
Version	0.1 Concept Draft
Date	August 5, 2026
Status	Design exploration; suitable for review, prototyping, and decomposition into RFCs
Primary objective	Define a programming language whose semantics, tooling, and source representation optimize safe maintenance by humans and AI systems
Normative vocabulary	MUST, MUST NOT, SHOULD, SHOULD NOT, and MAY are used to distinguish required, recommended, and optional behavior

SCOPE This document specifies a coherent target design, not merely a feature wishlist. Where features conflict, it chooses a default and records the tradeoff. Items marked “post-1.0” are deliberately excluded from the first implementation path.

Executive Summary

Sol is a statically typed, memory-safe, effect-aware, verification-capable programming language designed around the reality that software is continuously modified by teams containing both humans and generative models. Its central optimization target is not keystroke count. It is **preserving intent and correctness while changing code**.

The language combines a Rust-like safe implementation substrate, Koka/F*-style effect tracking, Dafny-style contracts, algebraic data types, explicit capabilities, structured concurrency, state-machine protocols, schema-aware data evolution, and a stable semantic representation for programmatic editing. It intentionally avoids unrestricted textual metaprogramming, unchecked exceptions, implicit nullability, ambient authority, and multiple equivalent syntactic idioms.

Sol uses a **two-tier correctness model**. The base language provides fast, predictable compilation with nominal types, ownership, exhaustive matching, explicit effects, and typed errors. A proof layer adds refinements, preconditions, postconditions, invariants, ghost state, and SMT-backed obligations. Normal application code does not need to become a theorem-proving exercise, but critical boundaries can be strengthened without changing languages or abandoning ordinary executable code.

The compiler exposes a canonical typed semantic graph, stable declaration identities, machine-readable diagnostics, constrained repair actions, proof caches, and behavior-change reports. These interfaces allow an AI system to make semantic patches against declarations and obligations rather than editing brittle text by line number. Humans retain authority over intent, architecture, accepted effects, and proof policy.

Key Decisions

Area	Decision
Execution	Ahead-of-time native compilation is primary; WebAssembly components and a lightweight VM/REPL are first-class secondary targets.
Memory	Affine ownership with inferred borrowing and lexical/explicit regions; no tracing garbage collector is required for ordinary code.
Types	Nominal algebraic types, distinct newtypes, generics, traits, refinements, units, and limited dependent relationships.
Effects	Row-polymorphic effect sets and explicit capability parameters; public APIs expose normalized effects.
Failure	Typed <code>Result</code> for recoverable failures; <code>panic</code> is a declared bug effect and is forbidden in verified profiles.
Verification	Contracts generate proof obligations discharged by typing, abstract interpretation, SMT, or explicit lemmas; runtime checks are a configurable fallback.
Concurrency	Structured task scopes, cancellation propagation, race-safe sharing, actors/channels, and protocol-typed resources.
Metaprogramming	Typed derives and semantic transforms only; no unrestricted token macros in the safe language.
AI interface	Stable semantic IDs, canonical IR, structured diagnostics, semantic patches, and change-oriented compilation.
Compatibility	Explicit versioned schemas, API fingerprints, migrations, and capability-aware FFI boundaries.

Contents

1. 1. Vision, Scope, and Design Principles
2. 2. Language Overview and Canonical Source Model
3. 3. Type System and Data Modeling
4. 4. Ownership, Regions, and Resource Safety
5. 5. Effects, Capabilities, and Failure Semantics
6. 6. Contracts, Refinements, and Verification
7. 7. Concurrency, Protocols, Transactions, and Durability
8. 8. Modules, Schemas, Interoperability, and Runtime Model
9. 9. AI-Native Development and Semantic Editing
10. 10. Compiler Architecture, Toolchain, and Developer Experience
11. 11. Standard Library, Ecosystem, and Security
12. 12. Implementation Roadmap, Risks, and Open Questions
13. 13. Worked Reference Examples
14. Appendix A. Syntax and Grammar Sketch
15. Appendix B. Standard Effect Taxonomy
16. Appendix C. Diagnostic and Patch Schemas
17. Appendix D. Comparative Positioning
18. Appendix E. References and Design Influences

READING PATH Readers evaluating the concept should begin with Sections 1, 2, 5, 6, and 9. Compiler implementers should continue through Sections 10 and 12. The worked examples are intentionally normative enough to expose inconsistencies in the proposed syntax.

1. Vision, Scope, and Design Principles

1.1 Problem Statement

Most mainstream languages were designed around a human author entering text and a compiler determining whether the text can execute. Modern software development has a different bottleneck. Large programs are maintained through repeated modifications made under incomplete context, frequently across multiple repositories, schemas, services, and toolchains. Generative models amplify both the rate of change and the cost of ambiguity.

The most damaging coding errors are rarely failures to express an algorithm. They are failures to preserve hidden assumptions: whether a value can be absent, whether a function performs I/O, whether an operation is atomic, whether a resource escapes a scope, whether a retry is idempotent, whether an old reader can consume a new schema, or whether a refactor changed an observable behavior. In conventional languages, these facts often live in comments, naming conventions, framework behavior, or tribal knowledge.

Sol treats these facts as language-level semantics. A function is not fully described by input and output types; its declared effects, error variants, ownership behavior, contracts, cost constraints, and protocol transitions are also part of its interface. The compiler maintains a semantic graph that tools can query and modify without reconstructing meaning from text.

1.2 Product Vision

Sol should allow a competent developer to write ordinary services, systems tools, embedded control code, libraries, and WebAssembly components without requiring formal-methods expertise. The same codebase should permit a security-critical module to add stronger contracts and machine-checked proofs incrementally. Verification is therefore a gradient, not a separate language or all-or-nothing project decision.

The intended development experience is: state intent and constraints; implement within a narrow semantic envelope; compile to receive structured obligations; apply a semantic change; and inspect a behavior-change report. The language should make the safe, reviewable path easier than a clever path.

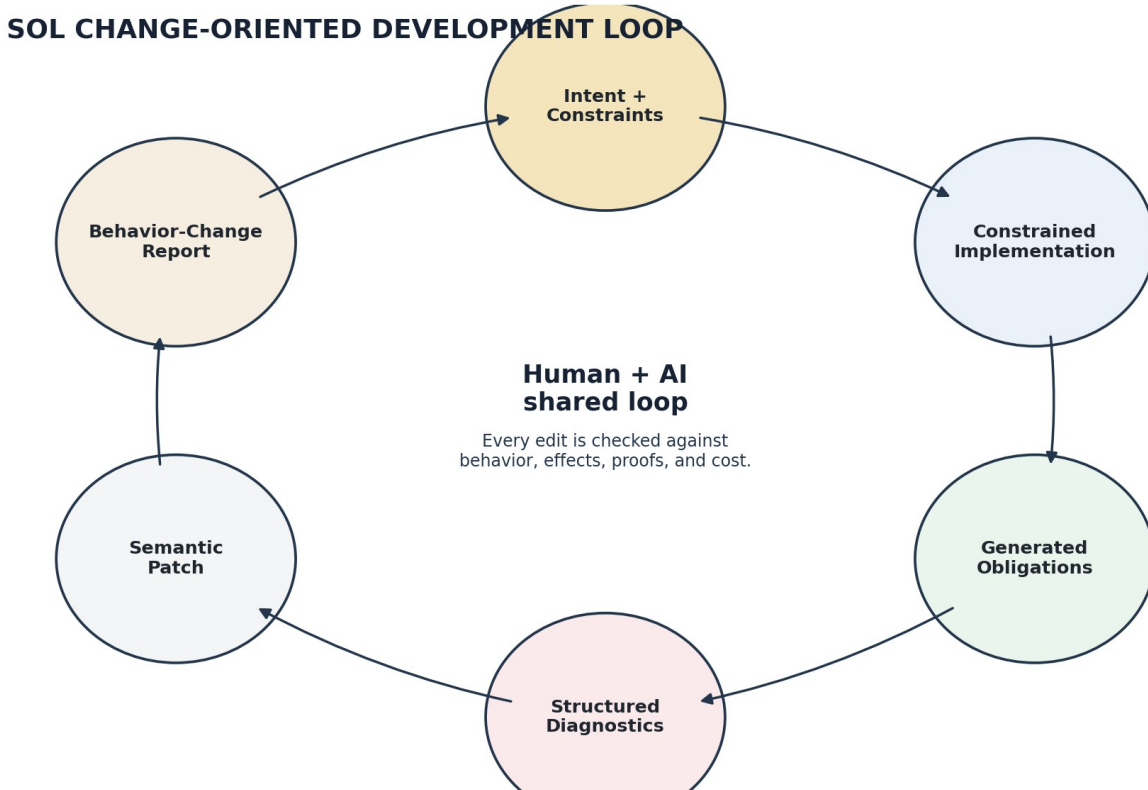


Figure 1. Sol optimizes the change loop, not merely the authoring step.

1.3 Primary Goals

- **Local semantic clarity.** A declaration should reveal nullability, mutation, effects, failure modes, resource ownership, and contracts without searching distant framework configuration.
- **Invalid-state resistance.** Domain states, protocols, and units should be representable as types rather than correlated booleans and comments.
- **Change safety.** Tooling should report behavioral, effect, schema, cost, and proof changes caused by a patch.
- **AI-legible structure.** The compiler should expose canonical semantic identities, machine-readable diagnostics, and valid repair classes.
- **Progressive verification.** Ordinary code should compile quickly; high-assurance code should be able to state and prove substantially stronger properties.
- **Predictable performance.** The implementation model should permit zero-cost abstractions, deterministic destruction, bounded allocation profiles, and low-level escape hatches that are explicitly quarantined.
- **Interoperability.** Sol should enter existing systems through C, WebAssembly components, generated bindings, and schema-defined service interfaces.
- **Canonicity.** Formatting and syntax should minimize semantically meaningless variation.

1.4 Non-Goals

- Sol is not intended to maximize terseness, code-golf expressiveness, or syntactic customization.
- Sol 1.0 will not expose unrestricted dependent types throughout ordinary programs. The proof language may be stronger than the everyday expression language.
- Sol will not guarantee that every terminating program can be proven automatically. Proof obligations may require annotations, lemmas, or an explicit decision to retain runtime checks.
- Sol will not hide distributed-systems failure behind transparent remote objects.
- Sol will not preserve source compatibility at the expense of semantic clarity during the pre-1.0 phase.
- Sol will not permit safe code to acquire filesystem, network, process, clock, random, database, or unsafe-memory authority implicitly.

1.5 Design Principles

Principle	Consequence
Intent is part of the program	Contracts, effect sets, protocol transitions, schema policy, and preservation constraints are versioned beside implementation.
Public interfaces are explicit	Inference may reduce local annotation, but exported types and effects have a single normalized printed form.
One obvious form	The formatter is mandatory; alternate declaration ordering, brace style, import style, and equivalent sugar are minimized.
Capability before convention	I/O and authority require values or declared effects, rather than ambient global access.
Proofs must compose	Verification is modular, cached, and bounded by declared interfaces; whole-program SMT is not the normal build model.
Escape hatches are visible	unsafe, unchecked FFI, raw memory, nondeterminism, and contract assumptions are explicit and auditable.
Diagnostics are an API	Compiler output is structured, stable, and carries machine-actionable repair alternatives.
Source is not the sole truth	A canonical typed semantic graph is a supported artifact, not an internal implementation accident.

1.6 Target Users and Domains

Sol's first-class target domains are backend services, command-line tools, systems components, embedded and real-time control, security-sensitive libraries, data-processing pipelines, and WebAssembly components. These

domains reward explicit effects and predictable resources while sharing enough abstractions to justify one language.

The initial language is not optimized for browser UI frameworks, highly dynamic scripting, symbolic mathematics, or GPU kernels, although it should interoperate with those ecosystems. A managed profile may eventually improve ergonomics for desktop and application code, but the baseline object model cannot depend on tracing collection.

2. Language Overview and Canonical Source Model

2.1 A Small Complete Example

Listing 1. Representative Sol declaration with domain types, effects, a transaction boundary, and postconditions.

```

module inventory.reservation

use commerce.OrderId
use inventory.{ItemId, Quantity}
use time.Instant

enum ReservationError {
  item_not_found(item: ItemId)
  insufficient_stock(
    item: ItemId,
    requested: Quantity,
    available: Quantity
  )
}

record Reservation {
  order: OrderId
  item: ItemId
  quantity: Quantity where quantity > 0
  created_at: Instant
}

public transaction reserve(
  order: OrderId,
  item: ItemId,
  quantity: Quantity where quantity > 0
) -> Result<Reservation, ReservationError>
effects {
  database.transaction<Inventory>
  clock.read
}
ensures {
  success => inventory[item].available
    == old(inventory[item].available) - quantity
  failure => inventory[item].available
    == old(inventory[item].available)
} {
  let stock = inventory.lock(item)
    .or_error(ReservationError.item_not_found(item))?

  require stock.available >= quantity else {
    return ReservationError.insufficient_stock(
      item = item,
      requested = quantity,
      available = stock.available
    )
  }

  stock.available -= quantity

  return Reservation {
    order
    item
    quantity
    created_at = clock.now()
  }
}

```

This example demonstrates the central surface model. Modules and imports are explicit. Data types are algebraic and nominal. Nullability is represented with `Option<T>`. Recoverable failure is a `Result`. The transaction boundary and authority are visible. Preconditions and postconditions are attached to the declaration. Mutation is confined to a compiler-known resource scope.

2.2 Lexical and Formatting Rules

- UTF-8 source is supported, but identifiers in public APIs SHOULD use a restricted Unicode profile to prevent confusable-name attacks.
- Keywords and standard-library identifiers are lowercase ASCII. Type names use UpperCamelCase; values, functions, fields, modules, and effects use snake_case.
- Blocks use braces. Semicolons are not used. Newlines separate statements after canonical formatting.
- Indentation is four spaces. Tabs are rejected by the canonical formatter.
- Trailing commas are required in multiline argument, field, and variant lists and removed in single-line forms.
- There is one formatter. Format output is stable for a given language edition and is treated as the canonical source representation.
- Comments are `//` and `/* ... */`; documentation comments are `///`. Nested block comments are supported.

CANONICAL SOURCE The compiler accepts a narrow grammar and `sol fmt` produces a single normalized spelling. This is not cosmetic: canonicity reduces diff noise, simplifies generated patches, and makes semantic hashes stable.

2.3 Declaration Ordering

Within a source file, declarations are formatted in the following order: module declaration; edition and feature declarations; imports; public type declarations; private type declarations; constants; capabilities; functions; protocol or transaction declarations; specifications; tests. Tools MAY preserve deliberate grouping through named regions, but arbitrary reordering is not a style preference.

Listing 2. Canonical file structure.

```
module billing.transfer
edition 2027

use identity.AccountId
use money.Money

public enum TransferError { ... }
public record TransferReceipt { ... }

const maximum_transfer = 100_000 USD

public function transfer(...) -> ...
effects { ... }
requires { ... }
ensures { ... } {
    ...
}

spec transfer { ... }

test "same-account transfers fail" { ... }
```

2.4 Expressions and Statements

Sol is expression-oriented, but it deliberately distinguishes effectful statements from pure expressions where doing so improves reviewability. `if`, `match`, and blocks produce values. Assignments, resource acquisition, task spawning, transaction control, and capability operations are statements. A block's final expression is its value.

Listing 3. Exhaustive matching and expression-valued control flow.

```

let label = match account.state {
  active => "Active"
  frozen(reason) => "Frozen: {reason}"
  closed(at) => "Closed at {at}"
}

let fee = if amount >= 1_000 USD {
  0 USD
} else {
  2 USD
}

```

2.5 Bindings and Mutation

- `let` creates an immutable binding.
- `var` creates a mutable local binding and is prohibited in pure expression contexts.
- Record values are immutable unless accessed through an exclusive mutable borrow or an owned modify scope.
- Mutation syntax is intentionally ordinary inside a scope whose exclusivity is already proven.

Listing 4. Explicit mutation scope.

```

let original = account

modify account {
  account.balance += deposit
  account.updated_at = clock.now()
}

// `original` remains an immutable value if Account is copied or persistent;
// it is not an alias to the modified storage location.

```

2.6 Functions and Methods

Free functions are the semantic primitive. Method-call syntax is permitted as type-directed sugar when the first parameter is marked `self`. Extension methods are namespaced imports, not global monkey patches. Functions are non-virtual unless dispatched through a trait value. Overload resolution is intentionally limited to avoid context-sensitive surprises.

Listing 5. Method syntax is a canonical projection of an ordinary function.

```

public function Money.add(
  self: Money<Currency>,
  other: Money<Currency>
) -> Money<Currency>
effects { pure } {
  return Money(self.minor_units + other.minor_units)
}

let total = subtotal.add(tax)

```

2.7 Modules and Visibility

A module is a semantic namespace and compilation unit, not necessarily a single file. Packages define a public module tree. Visibility levels are `private`, `module`, `package`, and `public`. Friend access is not supported; cross-package internal access requires an explicit capability or a separately versioned internal package.

Imports never change method lookup globally. Wildcard imports are rejected in library source and permitted only in REPL/test scopes. Every external name in a public signature is represented by its canonical fully qualified identity in the semantic graph, even when the source uses a short imported spelling.

3. Type System and Data Modeling

3.1 Type-System Strategy

Sol uses a layered static type system. The base layer is designed to remain decidable, fast, and predictable: nominal algebraic types, generics, traits, affine ownership, effect rows, and exhaustiveness. The refinement layer permits predicates over values and state. The proof layer permits ghost terms, propositions, lemmas, and solver-guided obligations. Crossing from stronger to weaker layers requires erasure or a verified abstraction boundary.

USABILITY DECISION Sol does not expose unrestricted full-spectrum dependent typing in routine application syntax. Value-dependent constraints are admitted where they produce clear obligations and stable tooling; advanced propositions remain in the proof sublanguage.

3.2 Primitive Types

Category	Types and behavior
Integers	Int8 through Int128, unsigned equivalents, pointer-sized IntSize/UIntSize, and arbitrary-precision BigInt. Overflow policy is explicit per operation/profile.
Floating point	Float16, Float32, and Float64 with IEEE-oriented semantics. Equality and ordering make NaN behavior explicit.
Decimal	Fixed-scale Decimal<Scale> and arbitrary decimal for finance/data exchange; no implicit conversion from binary floating point.
Boolean	Bool; domain booleans are discouraged when an enum communicates meaning.
Text	Text is immutable Unicode scalar text; Bytes is an immutable byte sequence. Indexing text by integer is not supported.
Characters	Rune is a Unicode scalar value. Grapheme operations are library abstractions with explicit locale/data dependencies.
Never/unit	Never has no values. Unit has one value and is printed ().

3.3 Records, Enums, and Tuples

Records are nominal products. Enums are nominal tagged unions whose variants may carry typed data. Pattern matching over a closed enum is exhaustive. Public enums are closed by default; an explicitly open enum allows unknown future variants and therefore requires a fallback case at use sites.

Listing 6. Nominal products, closed sums, and opt-in extensible wire enums.

```

record Address {
  street: Text
  city: Text
  region: Text
  postal_code: PostalCode
}

enum PaymentResult {
  approved(receipt: Receipt)
  declined(reason: DeclineReason)
  retryable(error: GatewayError, after: Duration)
}

open enum WireMessage {
  request(Request)
  response(Response)
}

```

Tuples are structural and intended for local grouping, not public domain models. Public function returns containing more than two independently meaningful fields SHOULD use a named record.

3.4 Distinct and Refined Types

A distinct type has the representation of its underlying type but is not implicitly interchangeable with it. A refined type additionally carries a predicate. Construction of a refined value generates an obligation or runtime validation, depending on context and profile.

Listing 7. Domain identity and value refinement.

```

type UserId = distinct Uuid
type AccountId = distinct Uuid

type Age = refined Int where 0 <= self <= 150

type EmailAddress = refined Text using email.is_valid

type NonEmpty<T> = refined List<T> where self.length > 0

```

Predicates used in widely instantiated types must be solver-friendly and free of observable effects. Refinements do not change runtime representation unless a validation tag or proof witness is explicitly retained.

3.5 Optionality and Absence

There is no implicit null value. Optional values use `Option<T>`, with variants `some(T)` and `none`. References obtained from unsafe or foreign code cannot enter safe Sol as nullable pointers; they must be wrapped, checked, or represented by a foreign handle type.

Listing 8. Explicit absence.

```

let nickname: Option<Text> = user.nickname

match nickname {
  some(value) => greet(value)
  none => greet(user.display_name)
}

```

3.6 Generics, Traits, and Constraints

Generics are reified in the semantic graph and normally monomorphized for native code. A backend MAY use dictionary passing or shared instantiations when ABI or code-size policy requires it. Trait conformance is explicit and coherent within a package dependency graph: a package may implement a trait for a type only when it owns the trait or the type.

Listing 9. Trait-bounded generic function.

```

trait Ordered {
  function compare(self, other: Self) -> Ordering
  effects { pure }
}

public function maximum<T: Ordered>(
  values: NonEmpty<View<T>>
) -> T
effects { pure } {
  ...
}

```

Traits may declare associated types, constants, and effects. Trait objects require explicit `dynamic Trait` syntax and define an ABI-stable dispatch boundary. Generic specialization is not selected through arbitrary user-written overlap rules.

3.7 Units, Dimensions, and Currency

Physical dimensions and currency identity are part of numeric types. The compiler performs dimensional algebra and rejects incompatible addition or comparison. Unit conversions are explicit but can be zero-cost when ratios are known at compile time.

Listing 10. Dimensional analysis and currency identity.

```

let distance: Meter = 100 meters
let elapsed: Second = 9.58 seconds
let speed: Meter / Second = distance / elapsed

function kinetic_energy(
  mass: Kilogram,
  velocity: Meter / Second
) -> Joule {
  return 0.5 * mass * velocity^2
}

let invoice_total: Money<USD> = 125.00 USD
let euro_total = exchange.convert(invoice_total, to = EUR)?

```

Dimension metadata is erased where the representation is unambiguous. Currency conversion is effectful because it depends on an exchange-rate source and timestamp; the type system prevents it from appearing as ordinary arithmetic.

3.8 Collections and Views

Type	Semantics
<code>Array<T, N></code>	Fixed-size contiguous owned array; N may be a compile-time natural.
<code>Vector<T></code>	Growable contiguous owned collection.
<code>List<T></code>	Persistent immutable list optimized for structural sharing.
<code>Map<K, V></code>	Ordered or hashed map is explicit in the concrete type; generic Map does not conceal ordering guarantees.
<code>View<T></code>	Read-only borrowed sequence with lifetime inferred from its source.
<code>Slice<T></code>	Contiguous borrowed range; mutable slices require exclusive access.
<code>Stream<T, E></code>	Effectful asynchronous sequence whose effect and error types are visible.

Collection operations distinguish eager allocation, lazy iteration, and effectful streaming. A transformation cannot silently change from allocation-free to allocating without affecting a declared cost contract or change report.

3.9 Pattern Matching and Exhaustiveness

Patterns may destructure records, enums, tuples, refined values, and protocol states. Guards are pure expressions. Exhaustiveness checking accounts for closed variants and simple refinements; more complex logical coverage may produce a proof obligation. A wildcard on a closed enum is discouraged in library code because it conceals newly added variants.

3.10 Type Inference Boundaries

Sol infers local variable types, generic arguments, borrow regions, and effect rows where the result is unambiguous. Public declarations, trait members, recursive functions, FFI boundaries, serialized fields, and persistent schemas require explicit canonical types. This balances authoring speed with interface legibility.

4. Ownership, Regions, and Resource Safety

4.1 Ownership Model

Every nontrivial resource has an owner. Values are affine by default: they may be consumed at most once unless their type is Copy or they are borrowed. Moving an owned value invalidates the source binding. Destruction is deterministic and follows scope exit unless ownership is returned, transferred, or placed in a longer-lived region.

The surface language aims to expose less lifetime syntax than Rust while retaining comparable safety. The compiler infers most borrow regions from lexical structure, effect boundaries, and escape analysis. Explicit region parameters appear when an API intentionally relates the lifetime of inputs and outputs.

Listing 11. Explicit region only where the output borrows from the input.

```
function first<'data, T>(
  values: View<'data, T>
) -> Option<View<'data, T>>
effects { pure } {
  ...
}
```

4.2 Borrow Categories

Form	Meaning
View<T> or borrow T	Shared immutable borrow. Any number may coexist while mutation is excluded.
inout T	Exclusive mutable borrow for the duration of the call or scope.
Owned T	The callee receives responsibility for eventual destruction or transfer.
shared T	Explicit reference-counted or runtime-managed ownership; thread-safety traits govern cross-task use.
handle<Resource>	Opaque capability-bearing resource with protocol-defined operations and deterministic close behavior.

4.3 Regions and Arenas

Lexical regions permit bulk allocation and destruction. A value allocated in a region cannot escape unless its type is proven independent of that region or it is copied/moved into a longer-lived owner. Region allocation is a standard capability and can be prohibited in heap-free profiles.

Listing 12. Region-scoped allocation.

```
region frame {
  let vertices = Vector<Vertex>.with_capacity(
    mesh.vertex_count,
    in = frame
  )

  build_vertices(mesh, into = vertices)
  renderer.draw(vertices.view())
}
// All frame allocations are reclaimed here.
```

4.4 Destructors and Cleanup

Types may define deterministic cleanup through the Drop trait. Cleanup cannot fail with a recoverable error because it may run during another failure path. Fallible shutdown is an explicit operation such as `close() -> Result<(), CloseError>`; Drop then performs only best-effort, nonblocking safety cleanup.

Destructors have a tightly constrained effect set. Network calls, arbitrary blocking, task spawning, and user-visible logging are forbidden unless the type is marked unsafe and the containing profile allows it.

4.5 Interior Mutability and Sharing

Interior mutability is represented by explicit synchronization or cell types whose effects are visible. Safe cross-task sharing requires Share; transfer requires Send. These traits are compiler-governed for primitive ownership types and may be implemented manually only with a proof or an unsafe block.

Listing 13. Synchronization is represented as authority and an effect.

```
let cache: shared RwLock<Map<Key, Value>> = RwLock.new({})

function lookup(key: Key) -> Option<Value>
effects {
  sync.lock<cache>
} {
  using read = cache.read()
  return read.get(key).copied()
}
```

4.6 Unsafe Code

unsafe is an effect and a lexical block. Unsafe code may dereference raw pointers, construct unchecked refined values, call unchecked foreign functions, or assert ownership facts the compiler cannot derive. Every unsafe block must declare the invariant it assumes and the safe property it establishes.

Listing 14. An unsafe block states its proof boundary.

```
unsafe because {
  assume buffer points to `length` initialized bytes
  establish returned slice is valid for region `packet`
} {
  return Slice.from_raw_parts(buffer, length, in = packet)
}
```

The compiler records unsafe blocks in the semantic graph, includes them in package audit reports, and invalidates dependent proof caches when their assumptions change.

4.7 Real-Time and Allocation Profiles

A package or function may declare a resource profile: `general`, `bounded`, `heap_free`, or `real_time`. Profiles constrain allocation, blocking, synchronization, panic, and destructor behavior. They are checked compositionally through effects and cost contracts.

Listing 15. Resource-constrained function.

```
function control_tick(
  state: inout ControllerState,
  input: SensorFrame
) -> ActuatorFrame
effects { pure }
resources {
  profile = real_time
  stack <= 2 KiB
  heap == 0 B
  worst_case_time <= 200 us
} {
  ...
}
```


5. Effects, Capabilities, and Failure Semantics

5.1 Why Effects Are in Function Types

Input and output types do not describe whether a function reads the clock, uses randomness, writes a database, performs network I/O, acquires a lock, suspends, allocates, logs, panics, or invokes unsafe code. These behaviors affect determinism, security, testing, retries, caching, and operational cost. Sol therefore includes an effect row in every function computation type.

Pure functions are the simplest case. An effectful function declares a normalized set. Local effect inference is permitted, but exported signatures and trait requirements print effects explicitly. A caller must possess compatible capabilities or handle the effect through an effect handler.

Listing 16. Pure and effectful function types.

```
function calculate_total(
  cart: Cart,
  rules: PricingRules
) -> Money<USD>
effects { pure }

function submit_order(
  order: PendingOrder
) -> Result<OrderId, SubmitError>
effects {
  database.transaction<Orders>
  network.call<PaymentGateway>
  clock.read
  random.secure
}
```

5.2 Effect Rows and Polymorphism

Effects form open, normalized rows. Generic higher-order functions may quantify over an effect row so they do not erase information about callbacks. Effect order in source is canonical and does not imply execution order.

Listing 17. Effect-polymorphic higher-order function.

```
function map<T, U, effects E>(
  values: View<T>,
  transform: function(T) -> U effects E
) -> Vector<U>
effects {
  E
  memory.allocate
}
```

Effects may carry parameters, such as a named database, network service, filesystem root, lock, or actor. This keeps authority granular enough for sandboxing and testing without making every low-level operation a distinct nominal effect.

5.3 Capabilities

An effect states what may happen; a capability states which authority enables it. Capabilities are values that can be passed, restricted, wrapped, and substituted in tests. Top-level application wiring supplies capabilities to the program. Library code cannot obtain ambient authority by importing a module.

Listing 18. Capability-bearing service boundary.

```

capability PaymentGateway {
  function authorize(
    request: AuthorizationRequest
  ) -> Result<Authorization, GatewayError>
  effects { network.call<Self> }
}

function checkout(
  gateway: capability PaymentGateway,
  order: Order
) -> Result<Receipt, CheckoutError>
effects {
  network.call<gateway>
} {
  ...
}

```

5.4 Effect Handling

Selected algebraic effects may be handled within a scope. A handler interprets operations and removes or transforms the corresponding effect from the outward type. Handlers are suitable for deterministic clocks, test randomness, tracing, state, retries, and lightweight domain effects. They are not used to disguise network distribution or violate ownership.

Listing 19. A test handles clock access deterministically.

```

handle clock.read with FixedClock(
  instant = "2026-08-05T12:00:00-05:00"
) {
  let receipt = issue_receipt(order)
  expect receipt.created_at == instant
}

```

5.5 Typed Errors

Recoverable failure uses `Result<T, E>`. Error types are algebraic and part of the public API. The propagation operator `?` is syntax for an explicit, type-checked early return; it does not invoke hidden stack unwinding. Error conversion requires a declared conversion function or matching variant relationship.

Listing 20. Typed error propagation.

```

function load_config(path: Path) -> Result<Config, ConfigError>
effects { filesystem.read<path.root> } {
  let bytes = filesystem.read(path)
  .map_error(ConfigError.read_failed)?
  let syntax = config.parse(bytes)
  .map_error(ConfigError.invalid_syntax)?
  return config.validate(syntax)?
}

```

5.6 Panic and Unreachable States

`panic` represents a program defect or violated internal invariant, not normal control flow. A function that may panic carries the panic effect unless the compiler proves the path unreachable. Verified and safety-critical profiles prohibit outward panic effects. Production applications choose a panic policy: abort, task termination, supervised actor restart, or host-defined trap.

Listing 21. Unreachability requires an obligation rather than an unchecked assertion.

```

unreachable because {
  match is exhaustive after `state.validate()`
}

```

5.7 Cancellation, Timeout, and Retry

Cancellation and timeout are typed outcomes, not invisible exceptions. An operation that can suspend is cancellation-aware unless explicitly marked noncancellable. Retry is a library combinator constrained by an Idempotent proof or an explicit compensation strategy.

Listing 22. Timeout and cancellation are explicit outcomes.

```
let result = within 500 ms {
  payment_gateway.authorize(request)
}

match result {
  completed(value) => value?
  timed_out => return CheckoutError.gateway_timeout
  cancelled => return CheckoutError.cancelled
}
```

5.8 Standard Effect Categories

Category	Examples
Observation	clock.read, environment.read, configuration.read
Nondeterminism	random.pseudo, random.secure, scheduler.nondeterministic
I/O	filesystem.read, filesystem.write, network.call, console.write, process.spawn
State	state.local, database.read, database.write, database.transaction
Concurrency	task.spawn, task.suspend, sync.lock, channel.send, actor.message
Resources	memory.allocate, memory.pin, blocking, gpu.dispatch
Failure	panic, cancel, timeout
Boundary	unsafe, ffi.call, host.intrinsic

6. Contracts, Refinements, and Verification

6.1 Verification Philosophy

Sol makes specifications executable enough to test and formal enough to prove. The compiler converts type refinements, contracts, invariants, pattern coverage, ownership facts, protocol transitions, and resource bounds into obligations. Different engines discharge different obligation classes: ordinary type checking, linearity analysis, abstract interpretation, symbolic execution, SMT solving, model checking for bounded protocols, or explicit proof terms.

The verification experience must remain modular. A package verifies against dependency interfaces, not their source implementations. Proof caches are keyed by semantic hashes of declarations, assumptions, solver configuration, and imported contracts. A formatting change cannot invalidate a proof; a changed effect or precondition can.

6.2 Preconditions and Postconditions

Listing 23. Function contract with ``requires``, ``ensures``, ``result``, and immutable update.

```
function withdraw(
  account: Account,
  amount: Money<USD>
) -> Account
requires {
  amount > 0 USD
  account.state is active
  account.balance >= amount
}
ensures {
  result.id == account.id
  result.balance == account.balance - amount
  result.state == account.state
}
effects { pure } {
  return account with {
    balance = account.balance - amount
  }
}
```

A call site must establish preconditions. An implementation must establish postconditions assuming the preconditions. `old(expression)` refers to entry-state values. Postconditions may branch on success/failure outcomes. Frame conditions are inferred from ownership and effects but can be strengthened with explicit preserves clauses.

6.3 Invariants and Loops

Loops require sufficient invariants when their correctness cannot be derived automatically. Termination is proven with a decreases measure in total functions. General functions may be partial and carry diverge; such functions cannot be used as proofs or in compile-time evaluation.

Listing 24. Loop invariant and termination measure.

```
var index = 0
var sum = 0

while index < values.length
invariant {
  0 <= index <= values.length
  sum == values[0..index].sum()
}
decreases {
  values.length - index
} {
  sum += values[index]
  index += 1
}
```

6.4 Ghost State and Erasure

Ghost declarations exist only for specification and proof. They cannot influence runtime branching, I/O, memory layout, or public ABI unless explicitly materialized through a proven reflection operation. The compiler erases ghost values and verifies noninterference.

Listing 25. Erased specification state and lemma.

```
ghost record TransferModel {
  total_before: Money<USD>
  total_after: Money<USD>
}

lemma transfer_preserves_total(
  before: Accounts,
  after: Accounts,
  amount: Money<USD>
) -> Proof<after.total() == before.total()> {
  ...
}
```

6.5 Proof Modes

Mode	Behavior
check	Type, effect, ownership, exhaustiveness, and lightweight contract checks. Complex obligations may become runtime checks or warnings according to policy.
verify	All declared proof obligations must be discharged or explicitly marked as assumptions.
verified profile	No runtime contract fallback, panic, unchecked arithmetic, unverified unsafe code, or unbounded nondeterminism.
audit	Produces an assumption ledger, unsafe inventory, proof coverage, solver provenance, and dependency-contract report.

6.6 Runtime Contract Fallback

Some refinements depend on external data or predicates outside the solver fragment. The programmer may choose `validate`, which retains a runtime check and yields a refined value on success, or `assume`, which requires an unsafe/proof-policy exception. Public APIs must state whether a refinement is statically proven, dynamically validated, or trusted from a boundary.

Listing 26. Dynamic validation versus explicit trust.

```
let email: EmailAddress = validate EmailAddress(input)
  .or_error(SignupError.invalid_email)?

let packet_length: PacketLength = unsafe assume_refined(raw_length)
```

6.7 Specifications and Generated Tests

`spec` blocks contain examples, properties, invariants, model functions, and generators. They compile into tests and proof obligations. The compiler derives boundary values from refined types and mutation operators, then reports generated coverage separately from handwritten tests.

Listing 27. Executable examples and universal properties.

```

spec normalize_username {
  example "trims and folds case" {
    input = " Alice.SMITH "
    output = "alice.smith"
  }

  property "idempotent" for all value: Text {
    normalize_username(normalize_username(value))
    == normalize_username(value)
  }

  property "no surrounding whitespace" for all value: Text {
    let normalized = normalize_username(value)
    normalized == normalized.trim()
  }
}

```

6.8 Solver Discipline

SMT automation is powerful but can become unpredictable. Sol constrains solver interaction through typed theories, explicit triggers only in advanced proof code, deterministic resource budgets, and proof diagnostics that identify expensive obligations. Builds record solver version and options. A proof that depends on a timeout-sensitive heuristic is reported as unstable rather than silently accepted.

- Quantifier-free arithmetic, algebraic data constructors, equality, and finite maps form the preferred automatic fragment.
- Nonlinear arithmetic, unrestricted quantification, floating-point equivalence, and concurrency invariants may require lemmas or specialized engines.
- Proof obligations are named and addressable; suppression occurs by obligation identity, not source line.
- A package may set maximum solver time and memory per obligation and for the aggregate build.

6.9 Cost and Resource Contracts

Complexity and resource bounds are specifications. Big-O declarations are verified against approved combinators and recurrence rules where possible; hard byte/time limits combine static analysis with target-specific certification runs. The change report treats weakened bounds as API changes.

Listing 28. Complexity and allocation contract.

```

function binary_search<T: Ordered>(
  values: SortedView<T>,
  target: T
) -> Option<Index>
effects { pure }
cost {
  time <= logarithmic(values.length)
  allocation == 0 B
}

```

7. Concurrency, Protocols, Transactions, and Durability

7.1 Structured Concurrency

Tasks are lexically scoped. A parent scope cannot complete while child tasks remain live unless ownership is explicitly transferred to a supervisor. Errors and cancellation propagate according to a declared policy. Detached fire-and-forget tasks are unavailable in safe application code.

Listing 29. Structured concurrent fan-out.

```
function load_dashboard(
  user: UserId
) -> Result<Dashboard, DashboardError>
effects {
  network.call<ProfileService>
  network.call<MessageService>
  network.call<AlertService>
  task.spawn
  task.suspend
} {
  concurrent cancel_on_error {
    profile = profile_service.load(user)
    messages = message_service.recent(user)
    alerts = alert_service.active(user)
  }

  return Dashboard {
    profile = profile?
    messages = messages.or_default([])
    alerts = alerts?
  }
}
```

A concurrent block creates a task scope and binds result handles. Policies include `cancel_on_error`, `collect_all`, and `supervise`. The compiler verifies that all results and resources are consumed before scope exit.

7.2 Race Safety

Ownership prevents unsynchronized shared mutable access. Data transferred into a task must implement `Send`. Shared immutable data must implement `Share`. Synchronization wrappers expose locking effects, poisoning policy, fairness, and cancellation behavior. Lock-order annotations and static cycle checks are available in verified profiles.

7.3 Actors and Channels

Actors provide isolated mutable state and typed asynchronous message handlers. Channels are linear endpoints whose protocol may be represented as a state machine. Mailbox capacity and overflow policy are part of the actor type because they affect reliability and memory bounds.

Listing 30. Typed actor with explicit mailbox policy.

```
actor InventoryActor(capacity = 1024, overflow = reject) {
  state inventory: Map<ItemId, Stock>

  message reserve(request: ReserveRequest)
    -> Result<Reservation, ReservationError> {
    ...
  }
}
```

7.4 Protocol Types and State Machines

Many APIs are legal only in particular states. Sol permits protocols to define state-indexed resources and transitions. A transition consumes one protocol state and returns another, so invalid sequencing is rejected at compile time. Dynamic protocols may erase state to a runtime tag at explicit boundaries.

Listing 31. State-indexed connection protocol.

```
protocol Connection {
  state disconnected
  state connected(socket: Socket)
  state authenticated(socket: Socket, session: Session)

  transition connect(
    from: disconnected,
    address: Address
  ) -> Result<connected, ConnectError>

  transition authenticate(
    from: connected,
    credentials: Credentials
  ) -> Result<authenticated, AuthError>

  transition query(
    from: authenticated,
    request: Query
  ) -> Result<(authenticated, Response), QueryError>

  transition close(
    from: connected | authenticated
  ) -> disconnected
}
```

7.5 Transactions

transaction is a language-level semantic boundary backed by a provider capability. It guarantees a consistent commit/rollback model, resource closure, and effect restrictions. The language does not assume all databases offer identical isolation; isolation, retry, and event semantics are explicit parameters.

Listing 32. Transaction policy is part of the declaration.

```
public transaction transfer_funds(
  source_id: AccountId,
  destination_id: AccountId,
  amount: Money<USD>
) -> Result<TransferReceipt, TransferError>
using database = AccountsDb
isolation = serializable
retry = safe_if_conflict
effects {
  database.transaction<AccountsDb>
  clock.read
} {
  ...
}
```

A transaction may emit commit-coupled events through an outbox effect. Direct external network calls are rejected by default because rollback cannot undo them. A saga or durable workflow is the appropriate abstraction for cross-system operations.

7.6 Durable Workflows and Sagas

A workflow declaration models steps that survive process restarts. Inputs, outputs, continuation state, retry policies, and compensations must be serializable and versioned. Workflow code is deterministic with respect to recorded events; direct clock, randomness, and network access are replaced by durable commands.

Listing 33. Durable saga with compensation.

```
workflow fulfill_order(order: OrderId) -> FulfillmentResult {  
  let payment = step authorize_payment(order)  
    retry exponential(max = 5)  
    compensate refund_payment(payment.authorization)  
  
  let shipment = step create_shipment(order)  
    retry exponential(max = 8)  
    compensate cancel_shipment(shipment.id)  
  
  return FulfillmentResult { payment, shipment }  
}
```

7.7 Atomicity and Concurrency Verification

For advanced verified code, atomic sections may open named invariants and manipulate ghost state. The everyday language exposes higher-level synchronization and protocol types; the proof sublanguage provides separation-logic-style reasoning for custom lock-free structures and concurrent algorithms. These proofs are deliberately isolated from routine business code.

8. Modules, Schemas, Interoperability, and Runtime Model

8.1 Packages and Editions

A package contains a manifest, module tree, feature declarations, target profiles, dependency locks, and policy files. Language editions stabilize syntax and core semantics. A new edition may change defaults or reserve keywords but does not silently reinterpret an older package.

Listing 34. Package manifest sketch.

```
package "inventory-service" {
  edition = 2027
  version = "0.4.0"

  targets = [native, wasm_component]
  profile.release = verified

  dependency "sol-http" = "^0.3"
  dependency "postgres" = "^1.1"

  capabilities {
    allow network.call<PaymentGateway>
    allow database.transaction<InventoryDb>
    deny process.spawn
  }
}
```

8.2 Stable Semantic Identities

Every exported declaration receives a stable semantic ID. By default the compiler derives it from package identity, module path, declaration kind, and an explicit evolution token. A declaration may supply an annotation when renaming or moving it while retaining identity.

Listing 35. Stable declaration identity.

```
@stable("billing.transfer.execute.v1")
public function transfer(...) -> ... { ... }
```

Stable IDs anchor semantic patches, API histories, proof caches, documentation links, telemetry schemas, and deprecation records. Reusing an ID for an incompatible declaration is rejected.

8.3 Schema Definitions and Evolution

Persistent data types carry explicit schema versions and compatibility policies. Migrations are typed functions whose totality and data-loss behavior can be verified. Generated encoders retain unknown fields when the wire format supports them, preventing destructive read-modify-write cycles across rolling upgrades.

Listing 36. Versioned schema and migration.

```
record UserProfile version 3 compatible backward {
  id: UserId @field(1)
  display_name: Text @field(2)
  locale: Locale = Locale.en_us @field(3)
  preferences: Preferences @field(4)
}

migrate UserProfile from version 2 to version 3 {
  locale = Locale.en_us
  preferences = Preferences.default()
}
```

Field identities are numeric or stable symbolic IDs independent of source ordering. Removing or changing a field produces a compatibility obligation. Database schema, event schema, and serialized-file policy use the same underlying model but distinct deployment adapters.

8.4 API Compatibility

The compiler computes a public semantic fingerprint including types, effects, errors, contracts, trait requirements, layout policy, and protocol states. Versioning tools classify changes as source-compatible, binary-compatible, wire-compatible, behavior-strengthening, behavior-weakening, or breaking. SemVer recommendations are generated from this classification but remain a package-owner decision.

8.5 Foreign Function Interface

The C ABI is the universal native boundary. Sol generates headers and safe wrappers for representable types. Complex algebraic values cross through generated handles or canonical interface records. FFI functions declare ownership transfer, nullability, thread policy, blocking, error mapping, and effects.

Listing 37. Explicit FFI contract.

```
extern "C" function zlib_compress(
  input: foreign_slice<Byte>,
  output: foreign_mut_slice<Byte>
) -> CInt
ffi {
  ownership input = borrowed
  ownership output = borrowed_exclusive
  nullability = forbidden
  thread = any
  blocking = false
}
effects { ffi.call<zlib> unsafe }
```

8.6 WebAssembly Components

WebAssembly components are a first-class distribution target because they provide interface-driven composition and capability-oriented hosting. Sol maps records, enums, results, options, resources, and worlds to the component model's canonical interfaces. Effects become host import requirements, enabling deployment-time capability checks.

8.7 Runtime Profiles

Profile	Runtime model
Native static	AOT executable or library, deterministic destruction, platform allocator or configured region allocator, optional async runtime.
Native hosted	AOT code with richer reflection metadata, package loader, tracing, and supervised actors.
WebAssembly component	No ambient host access; imports define capabilities and resources.
Embedded	No OS assumptions, selectable allocator, interrupt-safe subset, heap-free option.
Sol VM	Portable bytecode for REPL, tests, migration scripts, and fast edit-run cycles; not the performance reference.

8.8 ABI and Layout

Internal layout is unstable by default to permit optimization. `@repr(C)`, `@repr(transparent)`, and versioned stable ABI annotations opt into stronger guarantees. Public native ABI packages must freeze data layout, calling convention, panic policy, and allocator ownership. The compiler emits a machine-readable ABI manifest.

9. AI-Native Development and Semantic Editing

9.1 Design Objective

AI assistance is not implemented as a privileged compiler bypass. It is implemented as a first-class consumer of stable compiler APIs. The model proposes intent, code, proofs, or patches; the compiler determines semantic validity. The same interfaces are available to IDEs, refactoring tools, CI systems, and human-authored automation.

The key shift is from textual editing to **semantic editing**. Text remains the human-readable source of record, but modifications can target declarations, parameters, effects, control-flow nodes, contracts, and schema fields by stable identity. The formatter materializes the result back into canonical source.

9.2 Intent Blocks

An intent block records purpose, preservation requirements, forbidden outcomes, priorities, and operational assumptions. Free-form prose is permitted for explanation, but enforceable fields map to contracts, lints, or change-review rules. Intent does not become magically executable; it provides structured context and creates obligations where the semantics are defined.

Listing 38. Structured intent metadata.

```
intent transfer {
  purpose:
    "Move funds atomically between two accounts."

  preserve:
    - total money across both accounts
    - account identity
    - unrelated account fields

  forbid:
    - partial transfer
    - negative balance
    - transfer to the same account

  priority:
    correctness > observability > performance
}
```

9.3 Semantic Patch Language

The Sol patch language operates on the typed semantic graph. It can add or remove parameters, alter effects, insert statements at semantic anchors, update variants, rename stable declarations, and attach preservation constraints. A patch is rejected if its target identity is ambiguous or if it relies on stale semantic fingerprints.

Listing 39. Semantic patch with stale-base protection and preservation constraints.

```

patch billing.transfer::transfer {
  require base_fingerprint = "sha256:9e4d..."

  add parameter memo: Option<Text> after amount

  replace effect database.write<Accounts>
    with database.transaction<Accounts>

  insert before event FundsTransferred {
    audit.record_transfer_attempt(
      source = source_id,
      destination = destination_id,
      amount
    )
  }

  preserve {
    all existing error variants
    atomicity
    behavior except optional memo support
  }
}

```

9.4 Machine-Readable Diagnostics

Every compiler diagnostic has a stable code, semantic subject, source spans, violated rule, related declarations, proof context, and zero or more repair candidates. Human prose is a presentation layer over this structure. Repair candidates are constrained classes, not automatically applied guesses.

Listing 40. Structured effect diagnostic.

```

diagnostic {
  code: "SOL-EFFECT-014"
  severity: error
  subject: "billing.transfer.transfer"

  violation: {
    attempted_effect: "network.call<AuditService>"
    declared_effects: [
      "database.transaction<Accounts>",
      "clock.read"
    ]
  }

  valid_repairs: [
    add_effect("network.call<AuditService>"),
    inject_capability("AuditSink"),
    move_operation_to_caller
  ]
}

```

9.5 Change-Oriented Compilation

`sol check-change` compares two semantic graphs and reports externally observable differences. It understands more than source diffs: changed effects, weaker postconditions, stronger preconditions, altered error variants, schema compatibility, protocol transitions, allocation behavior, proof assumptions, and test obligations.

Listing 41. Behavior-change report.

```

$ sol check-change HEAD~1..HEAD

Public behavior:
+ Transfer requests accept an optional memo.

Effects:
! billing.transfer.transfer now calls AuditService.

Contracts:
= Atomicity preserved.
= Existing errors preserved.

Cost:
! Worst-case network calls increased from 1 to 2.

Verification:
842 obligations proved
3 new obligations proved
1 obligation unresolved:
  audit failure cannot make a committed transfer appear failed

```

9.6 Context Bundles

A tool can request a bounded context bundle for a task. The compiler selects the target declarations, normalized signatures, relevant intent, contracts, callers/callees, effect providers, protocol states, diagnostics, and recent semantic changes. This avoids sending an entire repository or relying on arbitrary text-window retrieval.

- Context bundles are deterministic and hash-addressed.
- Sensitive source may be replaced by interface summaries when implementation access is unnecessary.
- The bundle records omissions so an agent knows which assumptions are unsupported.
- Generated edits cite semantic subjects and base hashes, enabling safe replay or rejection.

9.7 Trust and Approval Model

Sol does not assign trust based on whether code was human- or AI-authored. Trust derives from review policy, verified properties, test coverage, unsafe assumptions, dependency provenance, and signer identity.

Organizations may require additional approval for patches that add capabilities, weaken contracts, introduce unsafe blocks, change durable schemas, or alter resource bounds.

9.8 Canonical Intermediate Representation

The supported semantic IR is readable, serializable, and versioned. It represents resolved identities, types, effects, contracts, regions, control flow, and proof obligations. Tools may consume either a compact binary form or a canonical textual form. The source language and IR are not isomorphic: comments and selected presentation choices remain source metadata, while implicit types and inferred effects become explicit in IR.

Listing 42. Human-readable Sol semantic IR sketch.

```
function billing.transfer.transfer {
  stable_id "billing.transfer.execute.v1"

  input source: identity.AccountId
  input destination: identity.AccountId
  input amount: money.Money<USD> refined greater_than_zero

  output Result<TransferReceipt, TransferError>

  effects {
    database.transaction<Accounts>
    clock.read
  }

  obligations {
    precondition same_account_forbidden
    invariant total_balance_preserved
    postcondition receipt_matches_commit
  }
}
```

10. Compiler Architecture, Toolchain, and Developer Experience

10.1 Compiler Pipeline

The compiler is implemented as a set of deterministic stages with cacheable artifacts. Parsing produces a lossless syntax tree. Name resolution and macro-free elaboration produce a canonical AST. Type, effect, and ownership checking produce typed HIR. Contract elaboration produces obligations. The semantic graph and Sol IR are then lowered into an ownership-explicit MIR and target backends.

SOL COMPILER AND VERIFICATION PIPELINE

The semantic graph is the stable interface for tooling, proof caching, and AI-assisted edits.

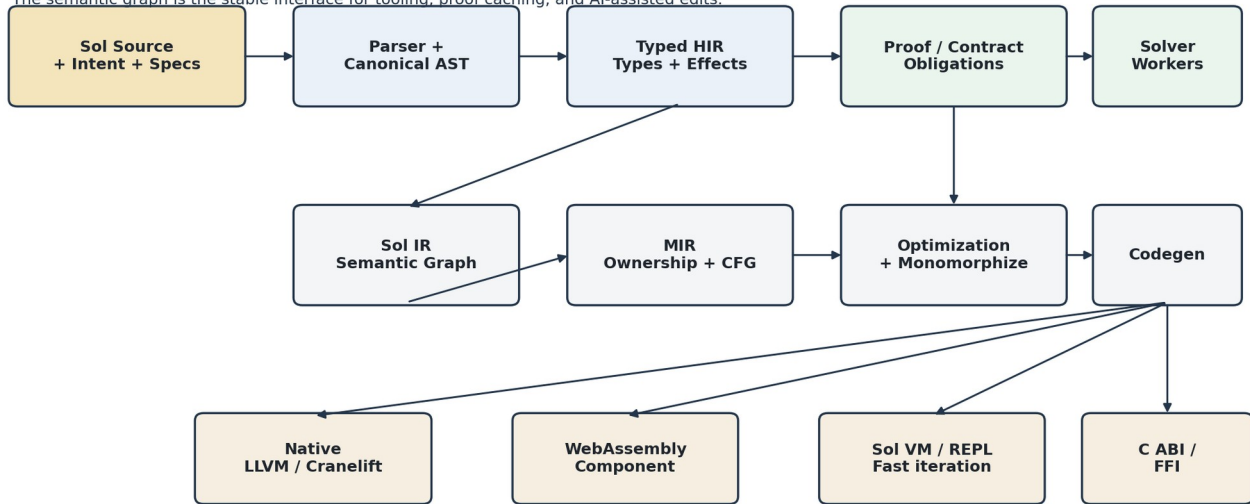


Figure 2. Proposed compiler and verification architecture.

10.2 Front End

- Incremental, error-recovering parser with a lossless syntax tree for IDE and formatter use.
- Edition-aware lexer and parser with no user-defined token macros.
- Deterministic name resolution and explicit import graph.
- Constraint-based type/effect/region inference limited by stable diagnostics and predictable worst-case behavior.
- Exhaustiveness and protocol-state checking before proof generation.

10.3 Verification Engine

The verifier normalizes contracts into a typed logical IR. It partitions obligations by theory and dispatches them to lightweight analyzers or solver workers. Verification runs in isolated processes with deterministic budgets. Proof results include assumptions, model counterexamples where available, and dependency hashes.

Counterexamples are mapped back to source-level values and protocol traces. An unresolved obligation is not summarized merely as “SMT timeout”; the diagnostic identifies the relevant path, quantifiers/theories involved, and recommended decomposition such as adding a loop invariant or lemma.

10.4 Sol IR and MIR

Sol IR is a high-level, typed, effectful semantic representation suitable for tooling and transformations. MIR is lower-level SSA/control-flow form with explicit moves, borrows, drops, regions, unwind/panic policy, and target-independent layouts. Proofs attach primarily to Sol IR; memory safety and optimization validation attach to MIR.

An MLIR-inspired multi-level representation is attractive because Sol-specific operations can remain explicit until enough semantic information has been used, while lowerings can target LLVM, Cranelift, WebAssembly, or specialized embedded pipelines. The design does not require adopting MLIR wholesale, but it adopts the principle that source, human-readable IR, in-memory IR, and serialized IR should describe the same semantic content.

10.5 Backends

Backend	Role
LLVM	Primary optimizing AOT backend for mature native targets and link-time optimization.
Cranelift	Fast development builds, JIT/REPL, and a simpler code-generation path for early compiler bring-up.
WebAssembly	Portable sandboxed components with capability imports and canonical interfaces.
C	Bootstrap/debug backend and integration path; not the preferred performance or semantics reference.
Sol VM	Fast tests, compile-time execution, migration validation, and deterministic sandboxing.

10.6 Incremental Compilation

Incremental keys are semantic rather than textual where possible. Separate caches exist for parsing, name resolution, public interfaces, monomorphizations, proof obligations, and code generation. A private implementation change should not invalidate downstream type checking if the exported semantic fingerprint is unchanged.

10.7 Command-Line Tool

Listing 43. Unified `sol` toolchain surface.

```
sol new service inventory
sol fmt
sol check
sol verify --profile critical
sol test --derive-boundaries changed
sol run
sol build --target native --release
sol build --target wasm-component
sol check-change origin/main..HEAD
sol explain SOL-EFFECT-014
sol audit --unsafe --capabilities --proofs
sol patch apply change.solpatch
```

10.8 Language Server and IDE

- Semantic completion including effects, capabilities, protocol states, and valid constructors.
- Inline obligation status and counterexamples.
- Refactors implemented as semantic patches and previewed as behavior changes.
- Call hierarchy annotated by effects and capability requirements.
- Ownership visualization and region explanations only when inference fails or the user requests them.
- Generated tests and proof holes surfaced separately from production code.
- Stable diagnostic protocol shared with CI and agent tooling.

10.9 Build Reproducibility

Build manifests pin compiler edition, package graph, target, proof engines, solver versions, build scripts, generated-source hashes, and capability policy. Build scripts execute as WebAssembly components with

declared filesystem/environment/network capabilities. Arbitrary host shell execution is not the default package model.

10.10 Documentation

API documentation is generated from signatures, effects, errors, contracts, examples, protocol diagrams, schemas, and intent. Documentation shows not only how to call a function but what authority it requires, whether it suspends or allocates, which failures are recoverable, and which properties are proven versus runtime-checked.

11. Standard Library, Ecosystem, and Security

11.1 Standard Library Principles

The core library is small, stable, and freestanding. Hosted functionality is divided into capability-oriented packages. APIs avoid hidden locale, clock, environment, filesystem, allocator, and executor dependencies. Algorithms expose complexity and allocation behavior where practical.

11.2 Library Layers

Layer	Contents
core	Primitive traits, algebraic types, numeric operations, text/bytes views, ownership helpers, and compile-time facilities.
alloc	Vectors, maps, strings, reference counting, arenas, and allocator interfaces.
std	Filesystem, networking, process, clock, random, concurrency, serialization, logging, and platform abstractions.
verify	Propositions, proof combinators, model collections, solver theories, and ghost helpers.
component	WebAssembly component interfaces, resources, worlds, and host adapters.
embedded	No-OS abstractions, volatile memory, interrupts, fixed-capacity collections, and hardware capability traits.

11.3 Text, Time, and Randomness

Text APIs distinguish bytes, Unicode scalar values, grapheme clusters, normalization, and locale-sensitive behavior. Time distinguishes monotonic duration, civil date/time, UTC-like instants, and timezone databases. Randomness distinguishes deterministic pseudo-random generators from secure entropy. These distinctions prevent convenient APIs from hiding data dependencies or nondeterminism.

11.4 Networking and Services

Network APIs are async and cancellation-aware by default. DNS, TLS roots, proxies, and credential sources are capabilities. Service clients are generated from schema definitions and preserve typed errors, deadlines, idempotency, and retry policies rather than collapsing everything into generic transport exceptions.

11.5 Logging, Metrics, and Tracing

Observability is an effect but can be handled by no-op, buffered, test, or production sinks. Structured events use stable schema IDs. Logging from pure functions is forbidden; a caller can instead consume returned diagnostic data or apply a tracing handler around an effectful computation.

11.6 Package Registry and Supply Chain

- Content-addressed package artifacts and reproducible source archives.
- Signed publisher identities and optional organization attestations.
- Machine-readable capability, unsafe, FFI, build-script, and proof metadata.
- Lockfiles pin exact dependency graphs and integrity hashes.
- Package policies can deny new capabilities or unsafe code transitively.
- Semantic API diffs are published with releases.
- Build scripts and procedural derives run in declared sandboxes.

11.7 Security Model

Sol's security model combines memory safety, capability-based authority, effect visibility, package sandboxing, schema validation, and auditability. It does not claim that types eliminate logic vulnerabilities. Instead, it attempts to make authority and assumptions small enough to review and constrain.

Threat	Language/tool response
Memory corruption	Safe ownership/borrowing; raw memory confined to audited unsafe blocks and FFI contracts.
Ambient authority	Capabilities required for filesystem, network, process, environment, database, clock, and randomness.
Dependency compromise	Signed/content-addressed packages, sandboxed builds, capability diffing, and policy gates.
Schema confusion	Nominal IDs, field identities, versioned migrations, unknown-field policy, and compatibility checks.
Unicode spoofing	Identifier normalization and confusable detection for public APIs.
AI-generated unsafe change	Semantic policy gates on new effects, unsafe blocks, weakened contracts, and proof assumptions.
Denial of service	Resource contracts, bounded profiles, mailbox policies, timeouts, and cancellation semantics.

11.8 Reflection and Serialization Safety

Runtime reflection is opt-in metadata generated per package. It cannot construct values that bypass invariants. Deserialization targets a validation boundary and produces typed errors; refined values are validated or accompanied by trusted schema proofs. Arbitrary object graphs and executable constructors are not a standard serialization feature.

12. Implementation Roadmap, Risks, and Open Questions

12.1 Recommended Implementation Strategy

The implementation should begin with a deliberately smaller core that proves the interaction among canonical syntax, algebraic types, ownership, effect rows, contracts, semantic identities, and structured diagnostics. Attempting full proof automation, durable workflows, all native targets, and a package ecosystem simultaneously would obscure whether the core model is coherent.

12.2 Phased Roadmap

Phase	Scope and exit criteria
0 - Executable language model	Formalize core syntax and semantics in an existing proof-oriented language; implement examples and resolve contradictions.
1 - Front end and interpreter	Parser, formatter, modules, ADTs, generics, pattern matching, typed errors, basic effects, canonical semantic graph, and VM interpreter.
2 - Ownership and native code	Affine types, borrow/region inference, deterministic drop, C FFI, Cranelift or C backend, and memory-safety validation.
3 - Contracts and SMT	Requires/ensures/invariants, refinement validation, obligation IR, solver workers, counterexamples, and proof caching.
4 - Concurrency and capabilities	Structured async, channels/actors, Send/Share, cancellation, effect handlers, and sandboxed capability hosts.
5 - Schemas and WebAssembly	Versioned schema compiler, migrations, component-model target, generated service bindings, and compatibility reports.
6 - AI/tooling interface	Semantic patch language, context bundles, behavior-change reports, policy gates, and agent SDK.
7 - Stabilization	Edition policy, package registry, standard library hardening, security audit, performance qualification, and 1.0 RFC process.

12.3 Minimum Viable Sol

The minimum viable language should resist becoming a generic “Rust with nicer syntax.” It must ship at least one differentiating vertical slice: explicit effects in public signatures, executable contracts, a stable semantic graph, structured diagnostics, and semantic change reports. Ownership alone is not sufficient to validate the Sol thesis.

- Canonical formatter and editioned parser.
- Records, enums, generics, traits, `Option`, `Result`, exhaustive match.
- Affine ownership and lexical borrowing for common patterns.
- Parameterized effect rows and capability injection.
- `requires`, `ensures`, `invariant`, `old`, `result`, and runtime/proof policies.
- Canonical semantic IR and stable IDs.
- JSON diagnostics and a basic semantic patch API.
- Native or WebAssembly executable output.

12.4 Major Risks

Risk	Mitigation
Feature interaction complexity	Define a small formal core and test every surface feature by lowering into it. Gate additions through RFCs with interaction

Risk	Mitigation
	matrices.
Verification brittleness	Modular contracts, stable obligation IDs, proof caching, deterministic solver budgets, and explicit proof-health metrics.
Ownership ergonomics	Aggressive but explainable region inference, persistent collections, scoped mutation, and targeted shared ownership.
Effect annotation fatigue	Infer locally, normalize publicly, support effect aliases, and keep parameterized effects at meaningful authority granularity.
Slow builds	Separate check/verify modes, incremental semantic hashes, parallel isolated solvers, and lightweight analyzers before SMT.
Tooling becomes model-specific	Publish stable generic protocols usable by IDEs, linters, CI, and multiple AI systems.
Ecosystem cold start	Excellent C/Wasm interop, generated bindings, package import tools, and a narrow high-value initial domain.
Unsound escape hatches	Small unsafe core, assumption ledgers, audited standard primitives, and verification of key compiler transformations.

12.5 Open Design Questions

1. Should effect handlers be available in the initial language, or should 1.0 expose capabilities and reserve general handlers until their interaction with ownership is proven?
2. How much dependent typing should be admitted outside ghost/proof code—compile-time naturals and protocol indices only, or a broader value-dependent fragment?
3. Should asynchronous suspension be a distinct `task . suspend effect` or a computation kind that contains other effects?
4. Can protocol state erasure and dynamic dispatch remain ergonomic without hiding runtime failure?
5. What proof language offers the best balance between readable Sol syntax and reuse of established theorem-proving infrastructure?
6. Should native packages standardize an ABI early, or rely on C/Wasm boundaries until the object model stabilizes?
7. How should semantic IDs evolve across forks, vendoring, and generated code?
8. Which resource-cost claims can be portable, and which must be target-certified?
9. Can a package registry reliably enforce transitive capability policy without making legitimate platform abstractions unusable?
10. What is the correct standard for an AI-generated repair candidate to be labeled safe, behavior-preserving, or proof-preserving?

12.6 Success Criteria

Sol succeeds if it measurably reduces maintenance defects and review effort on change-heavy systems. Evaluation should compare not just runtime speed or lines of code, but semantic diff accuracy, effect-leak prevention, contract coverage, repair success after compiler diagnostics, proof stability, refactor reliability, onboarding time, and the number of repository files an agent must inspect to make a correct change.

13. Worked Reference Examples

13.1 HTTP Handler with Explicit Authority

Listing 44. Service handler without ambient repository or authorization access.

```
module api.accounts

use http.{Request, Response, Status}
use identity.AccountId

public function get_account(
  request: Request,
  repository: capability AccountRepository,
  authorization: capability Authorization
) -> Result<Response, HandlerError>
effects {
  database.read<repository>
  authorization.check<authorization>
  tracing.emit
} {
  let account_id = request.path.parameter("account_id")
    .parse<AccountId>()
    .map_error(HandlerError.invalid_account_id)?

  authorization.require(
    request.principal,
    Permission.read_account(account_id)
  )?

  let account = repository.find(account_id)?

  return match account {
    some(value) => Response.json(Status.ok, value.public_view())
    none => Response.empty(Status.not_found)
  }
}
```

The handler's signature is sufficient to discover its operational authority. A test can replace both capabilities. A policy engine can reject a change that adds `network.call` or `database.write`. A cache wrapper can require proof that authorization is evaluated before returning cached protected data.

13.2 Idempotent Inventory Reservation

Listing 45. Transaction, idempotency intent, postconditions, and property test.

```

intent reserve_inventory {
  preserve:
    - available stock never becomes negative
    - retries do not reserve twice
    - stock update and reservation record commit atomically
}

public transaction reserve(
  order: OrderId,
  item: ItemId,
  quantity: Quantity where quantity > 0
) -> Result<Reservation, ReservationError>
using database = InventoryDb
isolation = serializable
effects {
  database.transaction<InventoryDb>
  clock.read
}
ensures {
  success => reservations.contains(order, item)
  success => inventory[item].available
    == old(inventory[item].available) - quantity
  failure => inventory[item].available
    == old(inventory[item].available)
} {
  match reservations.find(order, item) {
    some(existing) => return existing
    none => continue
  }

  let stock = inventory.lock(item)?

  require stock.available >= quantity else {
    return ReservationError.insufficient_stock(
      requested = quantity,
      available = stock.available
    )
  }

  stock.available -= quantity

  let reservation = Reservation {
    order
    item
    quantity
    created_at = clock.now()
  }

  reservations.insert(reservation)
  emit commit InventoryReserved(reservation)
  return reservation
}

spec reserve {
  property "retry is idempotent" for all valid request {
    let first = reserve(request)
    let second = reserve(request)
    expect second == first
    expect inventory_change(second_call) == 0
  }
}

```


13.3 Embedded Controller

Listing 46. Heap-free real-time function with physical units and bounded result.

```

record ControllerState {
  integral: Float32
  previous_error: Float32
}

function control_tick(
  state: inout ControllerState,
  target: Celsius,
  measured: Celsius,
  dt: Second
) -> DutyCycle
requires {
  0 s < dt <= 100 ms
}
ensures {
  0 percent <= result <= 100 percent
}
effects { pure }
resources {
  profile = real_time
  stack <= 512 B
  heap == 0 B
  worst_case_time <= 50 us
} {
  let error = target - measured
  state.integral = clamp(
    state.integral + error.value * dt.value,
    -1000.0,
    1000.0
  )

  let derivative = (error.value - state.previous_error) / dt.value
  state.previous_error = error.value

  return DutyCycle.clamped(
    kp * error.value + ki * state.integral + kd * derivative
  )
}

```

13.4 Protocol-Typed Database Session

Listing 47. Legal lifecycle encoded in resource state.

```
protocol Session {
  state disconnected
  state connected(connection: DbConnection)
  state transaction_open(connection: DbConnection, tx: Tx)

  transition connect(
    from: disconnected,
    config: DbConfig
  ) -> Result<connected, ConnectError>

  transition begin(
    from: connected
  ) -> Result<transaction_open, BeginError>

  transition commit(
    from: transaction_open
  ) -> Result<connected, CommitError>

  transition rollback(
    from: transaction_open
  ) -> connected

  transition close(
    from: connected
  ) -> disconnected
}

using session = Session.connect(config)?
using tx = session.begin()?
repository.update(tx, record)?
session = tx.commit()?
```

13.5 Semantic Refactor

Listing 48. Cross-cutting semantic refactor expressed by meaning, not textual search.

```
patch package inventory-service {
  select calls to legacy_clock.now

  replace with capability clock.now

  add capability parameter
    clock: capability Clock
    to nearest public entrypoint

  thread parameter through private callers

  preserve {
    returned timestamps
    error variants
    allocation behavior
  }

  require {
    no remaining effect environment.read<TZ>
    all tests use explicit clock handlers
  }
}
```

Appendix A. Syntax and Grammar Sketch

This appendix is illustrative rather than a parser-complete normative grammar. The authoritative grammar should be generated from the compiler source and versioned by edition. The sketch establishes the intended regularity and helps identify syntax interactions.

Listing 49. Condensed grammar sketch.

```

source_file      := module_decl edition_decl? use_decl* declaration*
module_decl      := "module" module_path NEWLINE
edition_decl     := "edition" INTEGER NEWLINE
use_decl         := "use" use_path NEWLINE

declaration      := visibility? annotation* (
    record_decl
  | enum_decl
  | type_decl
  | trait_decl
  | capability_decl
  | function_decl
  | protocol_decl
  | transaction_decl
  | workflow_decl
  | intent_decl
  | spec_decl
  | test_decl
)

function_decl     := "function" qualified_name generic_params?
                  ("(" parameter_list? ")")
                  "->" type
                  effect_clause?
                  requires_clause?
                  ensures_clause?
                  cost_clause?
                  resource_clause?
                  block

effect_clause     := "effects" "{" effect_entry* "}"
requires_clause  := "requires" "{" expression* "}"
ensures_clause   := "ensures" "{" expression* "}"

record_decl      := "record" TYPE_NAME version_clause? "{" field_decl* "}"
enum_decl        := "open"? "enum" TYPE_NAME "{" variant_decl* "}"
type_decl        := "type" TYPE_NAME generic_params? "="
                  ("distinct" type | "refined" type refinement)

statement        := let_stmt | var_stmt | assignment | return_stmt
                  | require_stmt | emit_stmt | using_stmt | expression_stmt
expression       := literal | path | call | block | if_expr | match_expr
                  | binary_expr | unary_expr | record_expr | lambda_expr

match_expr       := "match" expression "{" match_arm+ "}"
match_arm        := pattern guard? "=>" expression

result_type      := "Result" "<" type "," type ">"
option_type      := "Option" "<" type ">"
function_type     := "function" "(" type_list? ")" "->" type effect_clause

```

Appendix B. Standard Effect Taxonomy

The taxonomy below is a starting point. Concrete effects are parameterized by capabilities, resources, or roots. Aliases may group effects for domain APIs, but aliases expand to canonical rows in public semantic IR.

Effect	Meaning and notes
<code>pure</code>	No observable effects, deterministic for equal inputs, total unless separately marked diverge.
<code>diverge</code>	May not terminate. Excluded from proofs and compile-time evaluation.
<code>panic</code>	May terminate abnormally due to a violated internal invariant.
<code>memory.allocate</code>	May allocate through a specified allocator or region.
<code>memory.pin</code>	May create address-stable memory and therefore constrain movement/collection.
<code>clock.read</code>	Reads a wall, civil, or monotonic clock capability.
<code>random.pseudo</code>	Consumes deterministic pseudo-random state.
<code>random.secure</code>	Consumes secure entropy from a host capability.
<code>filesystem.read/write</code>	Access constrained by a filesystem-root capability.
<code>network.call/listen</code>	Outbound call or inbound listener for a named service/endpoint class.
<code>database.read/write/transaction</code>	Database operations parameterized by a repository/database capability.
<code>console.read/write</code>	Interactive terminal or stream access.
<code>environment.read</code>	Reads process/host environment; discouraged outside application wiring.
<code>process.spawn</code>	Creates or controls host processes.
<code>task.spawn/suspend</code>	Creates child tasks or yields/suspends execution.
<code>sync.lock</code>	Acquires a synchronization resource.
<code>channel.send/receive</code>	Uses a typed channel endpoint.
<code>tracing.emit</code>	Emits structured observability events.
<code>unsafe</code>	Relies on manually established memory/type invariants.
<code>ffi.call</code>	Crosses a foreign ABI boundary. Often paired with <code>unsafe</code> , but safe generated wrappers may hide it.

Appendix C. Diagnostic and Patch Schemas

C.1 Diagnostic Envelope

Listing 50. Proposed diagnostic JSON envelope.

```
{
  "schema": "sol.diagnostic/1",
  "code": "SOL-EFFECT-014",
  "severity": "error",
  "message": "undeclared effect network.call<AuditService>",
  "subject": {
    "stable_id": "billing.transfer.execute.v1",
    "kind": "function"
  },
  "locations": [
    {
      "file": "billing/transfer.sol",
      "start": { "line": 42, "column": 9 },
      "end": { "line": 42, "column": 31 },
      "role": "primary"
    }
  ],
  "facts": {
    "attempted_effect": "network.call<AuditService>",
    "declared_effects": [
      "database.transaction<Accounts>",
      "clock.read"
    ]
  },
  "repairs": [
    {
      "kind": "add_effect",
      "safety": "behavior_expanding",
      "requires_approval": "capability_policy"
    },
    {
      "kind": "inject_capability",
      "safety": "architecture_change"
    }
  ]
}
```

C.2 Patch Envelope

Listing 51. Proposed semantic patch exchange format.

```
{
  "schema": "sol.semantic-patch/1",
  "base_graph": "sha256:...",
  "operations": [
    {
      "op": "add_parameter",
      "target": "billing.transfer.execute.v1",
      "after": "amount",
      "parameter": {
        "name": "memo",
        "type": "Option<Text>"
      }
    }
  ],
  "preserve": [
    "public_errors",
    "atomicity",
    "existing_success_behavior"
  ],
  "author": {
    "kind": "tool",
    "name": "assistant",
    "signature": "optional-attestation"
  }
}
```

Appendix D. Comparative Positioning

Sol is intentionally synthetic. No current language provides the complete combination proposed here. The closest intellectual foundations are distributed across several languages and compiler systems. The comparison below describes design influence, not claims of direct compatibility.

System	Relevant strengths	What Sol adds or changes
F* + Pulse	Dependent/refinement types, effectful proof-oriented programming, SMT automation, ghost erasure, mutable-state and concurrency proofs.	A more conventional systems/application surface, canonical semantic editing, progressive proof modes, explicit change reports, and a unified production toolchain.
Dafny	Native specifications, contracts, ghost code, automated verification, familiar imperative syntax.	Ownership/resource safety, general effect rows, predictable native execution, capability security, and semantic patch protocols.
Rust	Ownership, borrowing, enums, exhaustive matching, typed recoverable errors, strong systems ecosystem.	Native effects and contracts, proof obligations, protocol declarations, schema evolution, and AI-oriented semantic interfaces.
Koka	Precise inferred effect types, algebraic handlers, effect polymorphism.	Ownership, nominal systems types, contracts/proofs, transactions, schemas, and production deployment profiles.
Idris 2	Dependent and quantitative types, linear resources, protocol-state examples, proof-driven programming.	SMT-first automation for routine contracts, canonical effects, native maintenance tooling, and a less dependent everyday language.
SPARK	Contracts, type predicates, proof of absence of runtime errors, high-assurance engineering discipline.	Algebraic types, ownership ergonomics, effect/capability rows, modern concurrency, and semantic agent tooling.
Pony	Actor model and reference capabilities for race-safe sharing.	General-purpose ownership outside actors, contracts/proofs, typed errors, transactions, schemas, and broader backend strategy.
MLIR	Multi-level IR approach, human-readable/in-memory/serialized forms, reusable transformations.	A language-specific stable semantic graph with contracts, effects, ownership, and patch identities as supported public APIs.

D.1 Closest Current Language

For the complete conceptual design, **F* with Pulse** is the closest current language family because it combines dependent/refinement types, effectful programming, automated proof, ghost erasure, mutable state, and concurrent separation logic. **Dafny** is closest to Sol's readable contract-oriented surface. **Rust** is closest to the proposed production implementation substrate. **Koka** most directly informs the effect-row and handler model.

Sol's differentiating claim is not any one feature. It is the integration of safe implementation, explicit authority, progressive proof, stable semantic representation, and change-oriented tooling into one coherent language experience.

Appendix E. References and Design Influences

The following primary sources informed the comparison and implementation discussion. They are not normative dependencies of this design.

[R1] [F* - A Proof-Oriented Programming Language](#) - Overview of F* as a dependently typed, effectful language with SMT and tactic-based proof automation.

[R2] [F* Tutorial: Computational Effects](#) - F* computation types, primitive effects, user-defined effects, divergence, and ghost computation.

[R3] [Pulse: Proof-Oriented Programming in Concurrent Separation Logic](#) - Embedded language for mutable state, concurrency, specifications, and proofs.

[R4] [Dafny Documentation and Reference Manual](#) - Specification constructs, static verification, compilation, framing, contracts, and ghost features.

[R5] [The Koka Programming Language](#) - Effect inference, effect polymorphism, effect handlers, and pure/effectful function types.

[R6] [The Rust Programming Language: Understanding Ownership](#) - Ownership and borrowing as a memory-safety model without garbage collection.

[R7] [Idris 2 Documentation: Linear Resources](#) - Quantitative/linear resource use and state-indexed protocol examples.

[R8] [SPARK User's Guide: Subprogram and Type Contracts](#) - Preconditions, postconditions, type predicates, and proof-oriented high-assurance development.

[R9] [Pony Tutorial: Reference Capabilities](#) - Reference capabilities for concurrency-safe sharing and transfer.

[R10] [MLIR Language Reference and Rationale](#) - Multi-level IR with textual, in-memory, and serialized forms for extensible compiler transformations.

[R11] [Craneflirt](#) - Code generator and CLIF IR suitable for fast development builds and JIT-style execution.

[R12] [WebAssembly Component Model](#) - Interface-driven, composable WebAssembly components and richer cross-language types.

Closing Statement

Sol is best understood as a proposal for a new programming interface between intent, implementation, verification, and change. Its syntax is deliberately conservative. Its novelty lies in making effects, authority, contracts, resource behavior, semantic identity, and change consequences normal parts of programming rather than optional layers reconstructed after the fact.

The most important implementation experiment is therefore not whether Sol can compile a benchmark. It is whether a human or model can modify a nontrivial system with less hidden context, receive a bounded set of meaningful compiler obligations, and prove—mechanically or operationally—that the requested behavior changed while unrelated behavior did not.

RECOMMENDED NEXT ARTIFACT Decompose this specification into a sequence of focused RFCs: Core Syntax and ADTs; Effects and Capabilities; Ownership and Regions; Contracts and Obligation IR; Semantic Graph and Stable IDs; Structured Diagnostics; and Semantic Patch Protocol.